because of a simple index error in an array. Further, the continuous integration scripts that run *Lean* and check the *Lean* code with external checkers are not formally verified. Such bugs can only be uncovered by empirical testing and user reports. This highlights a basic caveat when using interactive theorem provers. These tools check something highly specific, a proof, against a specification. Contributors are responsible for the correctness of everything else. This makes the role of organisers and maintainers especially important.

4.8. **The Role of organisers and maintainers.** As mentioned before, the correctness of the project depends on a lot of moving pieces, many of which cannot be guaranteed to be correct or functional by *Lean*. In this section, we give a brief description of the variety of tasks that need to be performed by maintainers. We wish to emphasize that the role played by maintainers is akin to that played by the principal investigator and senior postdoctoral researchers in a large experimental project, in that they need to understand the big picture and mathematical details of the project to a reasonable extent and be capable of making highly technical decisions, either themselves, on in consultation with subject experts. Further, they are likely to have limited time to get involved in highly specific details of the project, and while they might make technical contributions, most of their time will be spent managing and organising tasks for other contributors. The role requires a combination of mathematical research skills and capacity with software engineering tools.

The organisers and maintainers have several tasks in a project such as ours.

- They are responsible for monitoring the Zulip chat and onboarding new contributors.
- They are responsible for creating new tasks based on requirements and Zulip discussions, and ensuring that they are properly assigned.
- They are responsible for ensuring that all the project automation functions smoothly and step in when an issue is detected. It greatly helps to have a geographically distributed set of maintainers across timezones to help fix issues at any time of the day.
- They are responsible for reviewing all code, both *Lean* proofs and theorems, and non-*Lean* scripts. This includes ensuring that the automation to build and check proofs, compile the documentation and blueprint, and test and run scripts works smoothly. When necessary they must be willing to step in and build or repair the automation.
- They are responsible for reviewing and developing the basic definitions and theorems. As mentioned before, *Lean* takes definitions and theorems for granted. Thus maintainers need to be familiar with both the mathematical content and good ways of expressing this content in *Lean*. Being proficient in *Lean* internals is helpful for maintainers in identifying anti-patterns like the use of certain tactics that lead to trusting the *Lean* compiler, such as `native_decide`.
- Maintainers are responsible for helping contributors who might get stuck in a proof. Other contributors may also assist in such matters, but ultimately it is up to the maintainers to ensure progress.
- Experienced maintainers might also offer suggestions and guidance on how to produce shorter or more elegant proofs.
- They are responsible for ensuring that some basic standards are met in proof blocks that make proofs robust to upstream changes. For instance, non-terminal uses of the `simp` tactic must be replaced with the `simp only` tactic with an explicit list of

lemmas used. Otherwise, changes in the behaviour of upstream libraries can change how the tactic works and affect the correctness of the proof, when the *Lean* toolchain is updated.

- They are responsible for maintaining some record of the progress of the project. Projects on this scale can take a long time and it can become hard to remember how the project progressed. It would be extremely tedious to try to reconstruct real-time impressions long after the fact just from the GitHub commit history and Zulip chat archive. Thus it is extremely helpful if maintainers keep and publish regular logs of activities at a frequency that is proportionate to the scale of activity in the project. In our case, the main organiser maintained a daily log of activities during the busiest part of the project, which became less frequent as the progress rate slowed towards the end.

In this project, we were still learning a lot of the aforementioned lessons. A key learning from this project is that it really helps to have a maintainer structure in place before the project begins, rather than inventing one on the fly. Of course, new contributors can be onboarded as maintainers as necessary. But a small maintainer team in the initial stages can hamper proper review processes and result in suboptimal design choices in the *Lean* formalization that become hard to undo later; for instance, we developed a theory of free magmas before realizing that Mathlib already had this concept, but by that point we did not feel it worth the effort to refactor the existing code to use the Mathlib version. As one example, the initial list of equations were translated and put into one *Lean* file as opposed to several. This created excessively large *Lean* files which could have been managed better with a better file organisation.

In conclusion, we wish to emphasize that as projects scale, the administrative aspects of the project assume non-trivial importance. They require people who are proficient in at least part of the research topic, technical tools, and administrative matters. Setting these processes well in advance of announcing the project and inviting contributors should lead to a smoother project.

5. Counterexample constructions

In this section we collect the various techniques developed in the ETP to construct counterexamples to implications $\mathrm{E} \models \mathrm{E}'$.

5.1. **Finite magmas.** A finite magma $\mathcal{M}$ of size n can be assumed without loss of generality to have carrier $\{1, \dots, n\}$ and described by specifying the multiplication table $\diamond\colon \{1, \dots, n\} \times \{1, \dots, n\} \to \{1, \dots, n\}$. By generating a list of all the equational laws $\mathrm{E}j$, $j = 1, \dots, 4694$ satisfied by this magma, one can create refutations: if $\mathcal{M} \models \mathrm{E}j$ and $\mathcal{M} \not\models \mathrm{E}k$, then clearly $\mathrm{E}j \not\models_{\mathrm{fin}} \mathrm{E}k$ and hence also $\mathrm{E}j \not\models \mathrm{E}k$. (As mentioned previously, these statements were organized in Lean using the `Facts` statement.) It is feasible to brute force over all $\sum_{n=2}^{4} n^{n^2} \approx 4.3 \times 10^9$ non-trivial magmas of size at most 4 to obtain many refutations of this type. By performing brute force over all magmas up to size 4, a total of 13 632 566 implications (61.9% of all implications, and 96.3% of the false ones) can be refuted with 524 distinct magmas.

Table 1. Number of laws of order at most 4 whose smallest non-trivial model (if any) is of a given size.

Size of smallest non-trivial model	2	3	4	5	7	Trivial only
Number of laws of order 0	1					1
order 1	3					2
order 2	27					12
order 3	229	14	2			119
order 4	2876	18	12	14	2	1362
total (order ≤ 4)	3136	32	14	14	2	1496

Of these implications, 13 345 053 were refuted with magmas of size[7] 3, with the remaining 415 293 requiring magmas of size 4. Performing this search took 165 CPU-hours.

However, it is not feasible to exhaustively search over the $5^{5^2} \approx 3 \times 10^{17}$ magmas of size 5, even after quotienting out by isomorphism and symmetry (which roughly saves a factor of $5! \times 2 = 240$). Randomly sampling such magmas did not produce significant refutations, as random magmas of size 5 tended to satisfy few laws, and the set of laws covered were usually also exhibited by smaller magmas. A more fruitful approach was to randomly sample from magmas with additional properties that encouraged satisfiability of a greater set of laws. These included linear and quadratic magmas (discussed below), and cancellative magmas. On the other hand, some classes of magmas, such as commutative magmas, ended up producing a disappointingly small number of additional refutations.

For specific refutations, it was sometimes possible to locate a finite example with an ATP, particularly if one also imposed additional axioms (e.g., an idempotence axiom $x = x \diamond x$) that one suspected would be useful; see Section 7 for further discussion. For medium-sized magmas (of size $n = 5, 6, 7, 8$), this appeared to be a more efficient approach than brute force exhaustion of all such magmas.

It is a result of Kisielewicz [33] that every law En with $n \leq 4694$ is either equivalent to the singleton law E2, or else has a non-trivial finite model; in other words, the implications E$n \models$ E2 and E$n \models_{\text{fin}}$ E2 are equivalent for $n \leq 4694$. Our brute force search revealed that in the latter case there is always a model of size $2 \leq n \leq 5$, with the lone exception of E1286 (and its dual E2301), for which the smallest non-trivial finite model was of size 7, as presented in Example 5.2 below. In fact, most of the 4694 laws either only had trivial models, or had a size 2 model, as shown in Table 1.

Remark 5.1. The laws satisfied by a given finite magma $\mathcal{M}$ need not be finitely axiomatizable. The smallest example is the three-element magma $\{0, 1, 2\}$ with $1 \diamond 2 = 1$, $2 \diamond 1 = 2 \diamond 2 = 2$, and $x \diamond y = 0$ for all other x, y [54]. It was also shown in [55] that "almost all" magmas $\mathcal{M}$ (in a certain precise sense) are idemprimal[8], which implies that their laws

[7]For an earlier computer exploration of size 3 magmas, see [11].

[8]A magma is idemprimal if every idempotent function is expressed by a term. This is weaker than being primal (in which *every* function is expressible by a term), but stronger than being quasiprimal (in which the *discriminator* $f(a, b, c)$, defined to equal c if $a = b$ and a otherwise, is expressed by a term); quasiprimality